

Concorrência e transações

orm/concorrencia.py — duas transações disputando a mesma escala

O problema

O enunciado descreve o cenário: duas transações tentando escalar o mesmo residente para o mesmo dia, turno e unidade. A forma como a Etapa 1 tratava isso ilustra a falha.

routers/escala.py, Etapa 1

```
escala_conflito = db.execute(text("""
    SELECT 1 FROM escala
    WHERE id_unidade = :id_unidade AND dia_semana = :dia_semana
    AND turno = :turno AND id_residente = :id_residente;"""), {...}).first()

if escala_conflito:
    raise HTTPException(status_code=409, ...)

# ... e só depois o INSERT
```

Entre o SELECT e o INSERT existe uma janela. Duas requisições simultâneas podem passar as duas pela verificação antes de qualquer uma gravar, porque nenhuma enxerga a linha que a outra ainda não confirmou. A verificação prévia melhora a mensagem de erro no caso comum, mas não é o que garante a regra.

Como a simulação funciona

Cada cenário roda duas threads com sessões independentes. Uma barreira sincroniza o início, para que as duas cheguem juntas ao ponto de disputa em vez de uma terminar antes da outra começar. As sessões registram o que fazem com o instante relativo ao início, e é esse log que a aba Concorrência mostra.

As sessões executam SET LOCAL lock_timeout antes de qualquer coisa. Sem isso, uma thread bloqueada esperaria indefinidamente e travaria a requisição HTTP que dispara a simulação.

A combinação usada é o residente 15 no domingo à tarde, que o seed deixa livre. O que cada cenário cria é apagado no fim, então o banco volta ao estado anterior e a simulação pode rodar quantas vezes for preciso.

Cenário 1: apenas as restrições do banco

As duas sessões consultam, nenhuma encontra escala, e as duas tentam gravar. A segunda fica bloqueada no índice único até a primeira confirmar, e então falha.

Instante	Sessão	O que acontece
0,00s	A e B	transações abertas
0,01s	A e B	consultam: nenhuma escala encontrada
0,02s	A	INSERT enviado
0,02s	B	INSERT enviado, fica bloqueado no índice
0,03s	A	commit concluído

Instante	Sessão	O que acontece
0,03s	B	recusado: UniqueViolation

As duas consultas prévias disseram que o horário estava livre, e mesmo assim uma das gravações falhou. É o que mostra que verificar antes de inserir não substitui a restrição.

Cenário 2: bloqueio pessimista

Cada sessão pega um bloqueio na linha do residente antes de consultar. A segunda espera a primeira terminar e então enxerga a escala recém-criada.

orm/concorrencia.py

```
db.execute(select(Residente)
            .where(Residente.id_profissional == ID_RESIDENTE)
            .with_for_update()).scalar_one()

# daqui em diante, ninguém mais mexe neste residente até o commit
existe = db.execute(select(Escala).where(...)).first()
if existe:
    return # desiste com mensagem de negócio
```

O desfecho é diferente do primeiro cenário. Uma sessão grava e a outra desiste por regra de negócio, com mensagem legível, sem nenhuma violação de constraint. O custo é a espera: a segunda sessão fica parada até a primeira confirmar.

Cenário 3: bloqueio otimista

Ninguém bloqueia nada. As duas sessões carregam a mesma escala e alteram o turno. A coluna versao entra no WHERE do UPDATE.

SQL que o version_id_col gera

```
UPDATE escala SET turno = 'manha', versao = 2
WHERE id_escalas = 8 AND versao = 1
```

A primeira sessão a gravar muda a versão de 1 para 2. A segunda, que leu a versão 1, encontra zero linhas afetadas, e o SQLAlchemy conclui que alguém gravou no intervalo. O resultado é StaleDataError, que a rota traduz em 409.

Sem esse controle, a segunda gravação sobrescreveria a primeira sem aviso. O banco não recusaria nada, porque as duas alterações são válidas isoladamente; o problema é a primeira ter sido perdida.

Comparação

	Pessimista	Otimista
Mecanismo	SELECT ... FOR UPDATE	coluna de versão no WHERE
Quando detecta	antes de agir, esperando	na gravação, falhando
Custo no caso comum	espera e contenção	nenhum
Custo no conflito	nenhum, o segundo já vê o estado novo	refazer o trabalho
Serve para	conflito frequente e caro de refazer	conflito raro
Onde está no projeto	orm/concorrencia.py, cenário 2	Escala.version_id_col, ativo em PUT /orm/escalas/

A escolha depende da frequência do conflito. Escala de plantão é editada por poucas pessoas, e conflito real é raro, então o controle otimista ficou ativo nas rotas. O pessimista aparece na simulação, para comparação.

A restrição do banco continua necessária

Os dois mecanismos ficam na aplicação, e nenhum protege contra escrita feita por fora dela: um INSERT pelo psql, um script de importação, outra aplicação. A UNIQUE do esquema e o trg_check_sobreposicao_escala são a garantia que não depende de quem está chamando. Os locks servem para transformar a violação em mensagem útil, não para substituir a restrição.

Como executar

Pelo terminal, com o banco no ar:

Backend

```
python demo_concorrencencia.py
```

O script imprime os três cenários com os logs alinhados por instante e termina com código de saída zero quando os três se comportam como esperado. A mesma simulação está em POST /etapa2/concorrencencia/simular e na aba Concorrência da interface, onde cada cenário aparece com seu log e desfecho.

A execução leva alguns segundos. O cenário pessimista envolve espera real por bloqueio, e essa espera faz parte do que está sendo demonstrado.